

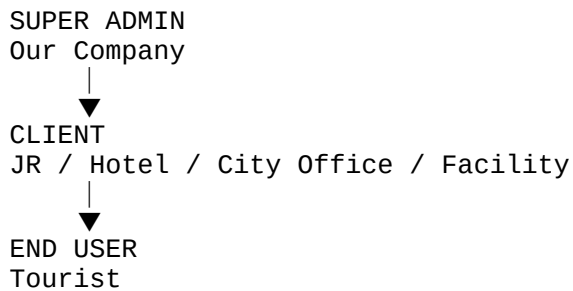
Tourist Map System — Full System Blueprint

1. Product Goal

The Tourist Map System is a configurable digital tourism platform designed for organizations such as:

- JR / railway operators
- Hotels
- Municipal and city offices
- Tourist information centers
- Shopping facilities
- Airports
- Museums and public facilities

The system has three user levels:



The first commercial development target will be:

STAGE 1
Client Admin
+
Android Tourist Signage
+
Tourist Smartphone QR Experience

After Stage 1 is stable:

STAGE 2
Advanced Client Features

Then:

STAGE 3
Super Admin
+
Widget / Feature Store
+
Platform Management

2. Core Product Concept

A client creates and configures its own tourist map from a web-based administration system.

The Android signage application reads the client's published configuration and displays a fullscreen interactive tourist map.

Example:



A tourist can select a place marker.

Example:

ヨドバシカメラ
マルチメディア京都

The Android signage opens a side information panel containing the place image, description, voice guide and QR navigation.

The tourist can then scan the QR code using their own smartphone and continue navigation from their current location.

3. System Users

User	Authentication	Purpose
Super Admin	Required	Manage entire platform
Client Admin	Required	Build and publish own maps
Tourist	None	Use public tourist signage
Tourist Smartphone	None	Receive destination/navigation

Tourists should never need to register.

4. Multi-Tenant Architecture

Every client receives a permanent unique customer ID.

Example:

```
customerId:  
cust_01HXYJRWEST
```

Each tourist map receives its own map ID.

```
mapId:  
map_01KYOTOSTATION
```

A customer can own multiple maps.

```
JR West  
|  
├─ Kyoto Station  
|  
├─ Osaka Station  
|  
└─ Kobe Station
```

The architecture therefore becomes:

```
Customer  
|  
├─ Map  
|   ├── Places  
|   ├── Categories  
|   ├── Events  
|   ├── Languages  
|   ├── Menu  
|   ├── Widgets  
|   └── Devices  
└─ Map
```

Even if the first version allows one map per customer in the UI, the database should support multiple maps from the beginning.

5. Recommended Technology Stack

Client Admin Web

Next.js
TypeScript
React
Responsive web application

Super Admin

Use the same web platform with role-based access.

/admin/*
/super-admin/*

Authentication

Firebase Authentication

Supported initially:

Email
Password

Future:

Google
Microsoft
SSO

Database

Cloud Firestore

File Storage

Firebase Storage

Used for:

Logos
Place images
Marker images
Event images
Uploaded audio
Theme assets

Backend

Recommended:

Firebase Cloud Functions

or:

Google Cloud Run

Cloud Run is preferable when backend functionality becomes larger.

Android

Kotlin

Jetpack Compose

Maps

Implement a provider abstraction.

Initial:

Google Maps

Architecture-ready:

Google Maps

Mapbox

MapLibre

Place Search

Google Places API

Voice Guide

Initial:

Text-to-Speech

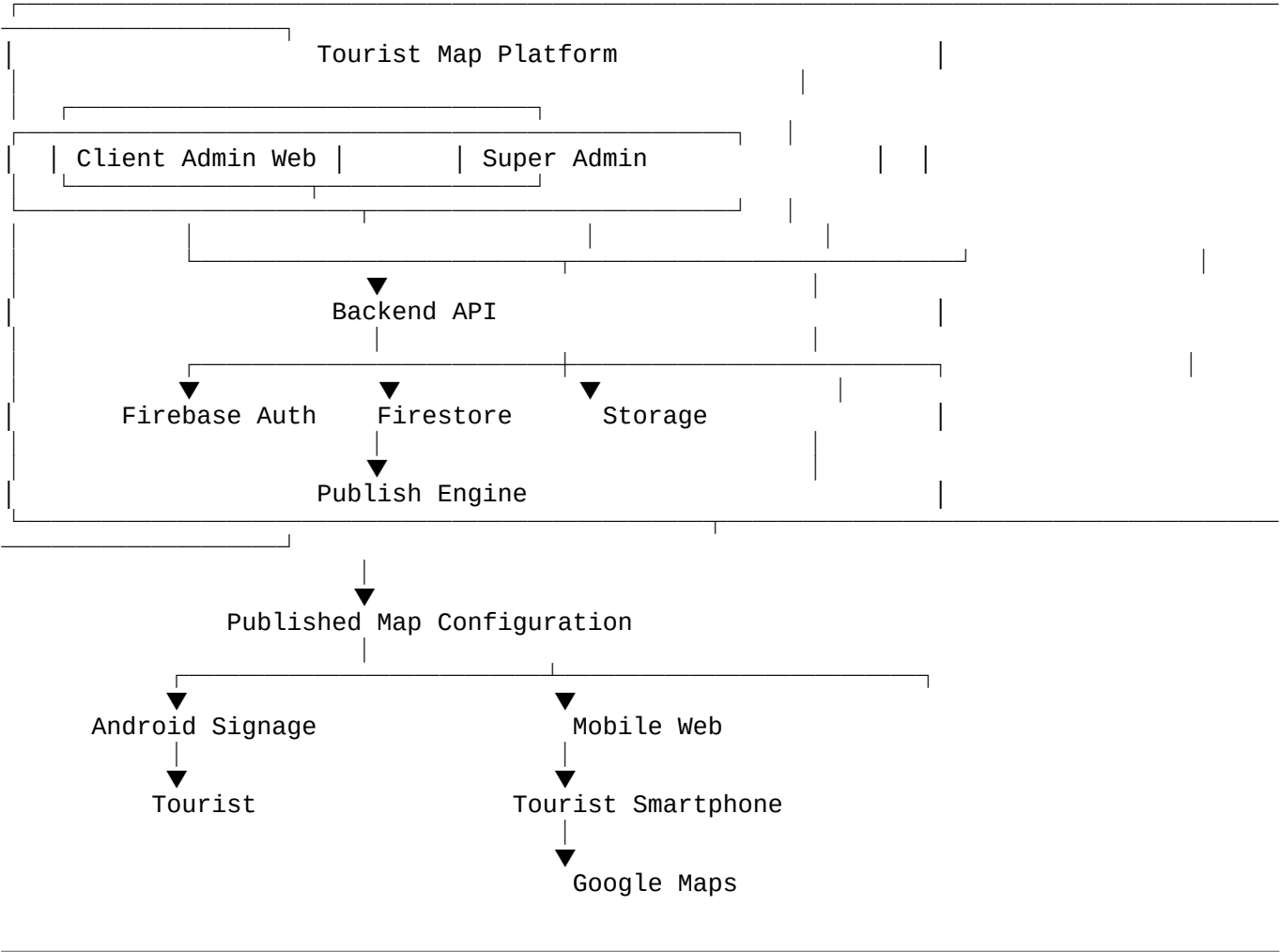
Future:

Uploaded recorded voice

AI generated premium voice

6. High-Level Architecture

INTERNET / CLOUD



7. Stage 1 Scope

Stage 1 proves the complete business workflow.

```
Client registration
↓
Client login
↓
Create tourist map
↓
Add places
↓
Configure categories
↓
Configure languages
↓
Customize appearance
↓
Preview
↓
Publish
↓
Android signage updates
↓
Tourist selects marker
↓
Voice Guide / QR
↓
Smartphone navigation
```

Stage 1 should be considered complete only when this entire workflow works on a physical Android signage device.

8. Stage 1A — Foundation

Build the project foundation first.

Client Registration

Registration page:

```
Company Name
Contact Name
Email
Username
Password
Confirm Password
```

[Create Account]

On successful registration:

```
Firebase User created
↓
customerId generated
↓
Client account created
↓
Default map created
↓
mapId generated
↓
Redirect to Client Dashboard
```

Client user record:

uid
customerId
role
displayName
email
createdAt
status

Role:

CLIENT_ADMIN

9. Client Dashboard

Recommended navigation:

Dashboard

Map

- Map Settings
- Map Area
- Places
- Categories

Content

- Events
- Live Cameras
- Media

Appearance

- Branding
- Theme
- Menu
- Marker Style

Languages

Voice Guide

QR Navigation

Widgets

Devices

Preview

Publish

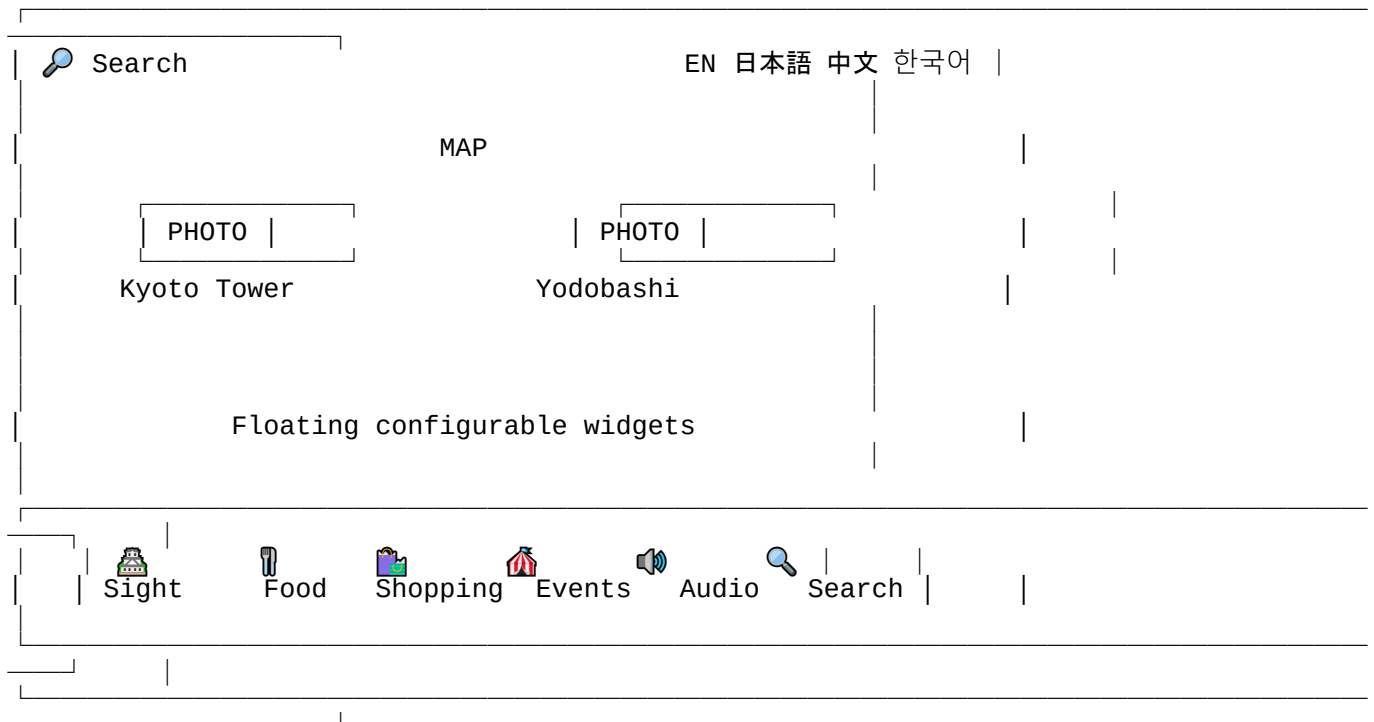
Account

During early Stage 1, disabled future features can be hidden.

10. Tourist Android Main Screen

The Android signage application runs fullscreen.

Recommended base design:



The map remains the dominant screen element.

11. Marker Design

Support multiple marker types.

DEFAULT_PIN
PHOTO
CUSTOM_IMAGE
CATEGORY_ICON

Primary Stage 1 marker:

PHOTO

Example:

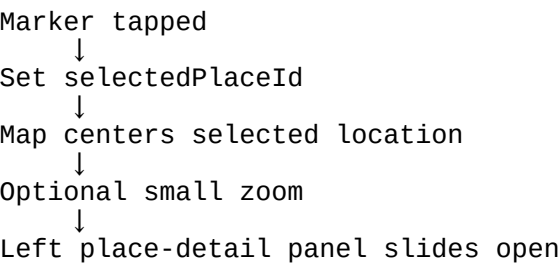


The number badge can eventually represent grouped nearby places.

Clustering can be added later.

12. Marker Selection Behavior

When a tourist touches a marker:



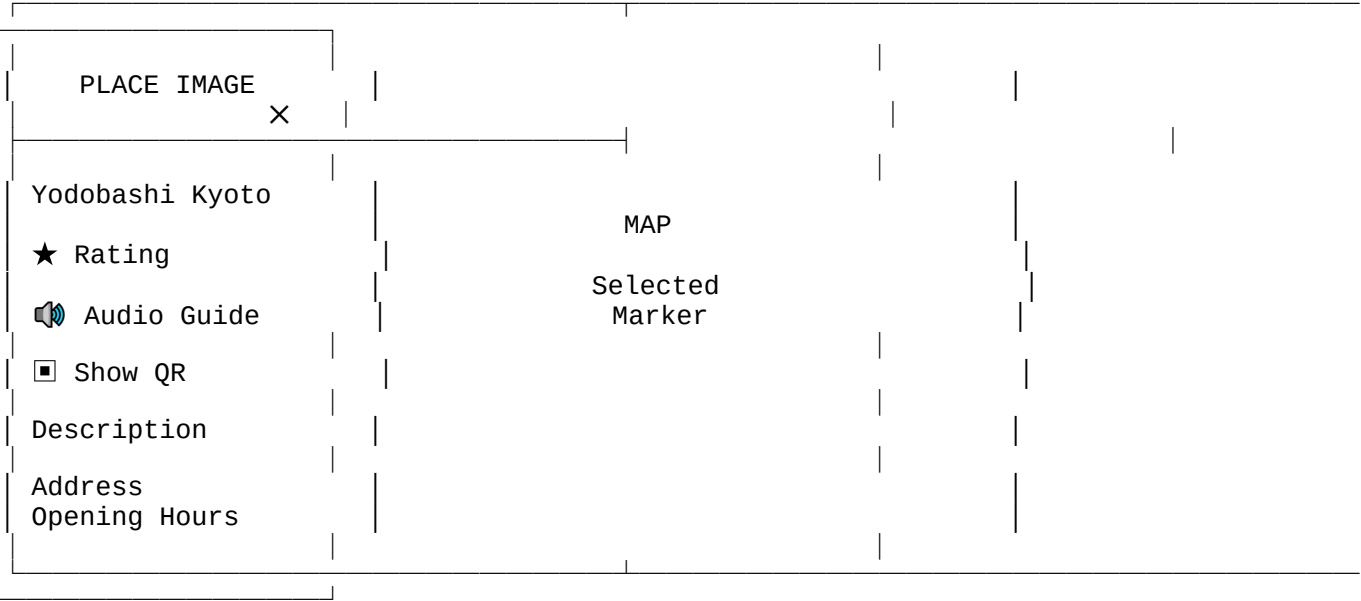
The user remains on the same map screen.

No full-page navigation.

13. Place Detail Side Panel

The side-panel design should follow the reference UI.

Desktop/signage landscape:



Recommended width:

35–40% screen width

Map:

60–65%

14. Side Panel Close Behavior

Panel closes when:

- X button pressed
- Empty map area touched
- Another marker selected
- Idle timeout reached
- Home/reset button activated

For public signage:

idleResetSeconds = configurable

Example:

90 seconds

After inactivity:

- Close panel
- Reset language
- Reset selected category
- Reset search
- Return map to default camera

15. Place Data Model

A place contains:

- placeId
- mapId
- customerId
- source
- googlePlaceId
- name
- categoryId
- latitude
- longitude
- mainImage
- gallery
- address
- phone
- website
- rating
- openingHours
- markerType
- markerImage
- status
- translations
- voiceGuide
- qrNavigation

displayOrder

createdAt
updatedAt

Source:

GOOGLE_PLACES
CUSTOM

16. Google Places Import

Client selects:

Add Place

Then:

Search Google Places
or
Create Custom Place

Google Place workflow:

Search:
ヨドバシカメラ 京都

↓

Search results

↓

Select result

↓

Import:
Name
Coordinates
Address
Photo
Google Place ID
Rating
Opening information

After import, the client can customize the content.

The Google source should never prevent custom information.

17. Custom Place Creation

Required fields:

Name
Category
Latitude
Longitude
Description

Image

Optional:

Address

Opening Hours

Website

Phone

Voice

Marker Image

QR behavior

Coordinates can be selected by clicking the admin map.

18. Category System

Categories should not be permanently hardcoded.

Default templates:

All

Sightseeing

Gourmet

Shopping

Events

A hotel could instead create:

Nearby

Restaurants

Shopping

Transportation

Hospitals

ATM

JR:

Tourism

Station Facilities

Food

Shopping

Hotels

Bus

Taxi

Client controls:

Name

Icon

Color

Enabled

Order

19. Bottom Menu Builder

The Android bottom menu should be generated from configuration.

Client Admin example:

Menu Builder

Sightseeing	Enabled	↑ ↓
Gourmet	Enabled	↑ ↓
Shopping	Enabled	↑ ↓
Events	Enabled	↑ ↓
Live Camera	Disabled	↑ ↓
Audio Guide	Enabled	↑ ↓
Search	Enabled	↑ ↓

Supported actions:

Enable
Disable
Rename
Reorder
Change icon

20. Multiple Languages

Each map has enabled languages.

Example:

English	Enabled
日本語	Enabled
中文	Enabled
한국어	Disabled
Français	Disabled

Map setting:

```
defaultLanguage = en
```

Tourist can switch using the header.

Example:

EN | 日本語 | 中文

21. Translation Data Model

Do not duplicate places per language.

Use:

```
translations:
  en:
    title
    shortDescription
    description

  ja:
    title
    shortDescription
    description

  zh:
    title
```

shortDescription
description

The same pattern should be used for:

Places
Categories
Events
Menus
Custom buttons
Announcements

22. Voice Guide

The voice guide uses the currently selected language.

Example:

Current UI language:
English

Tourist taps:
▣ Audio Guide

↓

English description

↓

TTS

↓

Speaker output

Initial implementation:

Text → Android / Cloud TTS

Future:

AI Voice
Uploaded MP3
Professional narration

Place voice configuration:

voiceMode:
TTS
UPLOADED_AUDIO
DISABLED

23. QR Navigation

The QR should not directly contain a permanent Google Maps URL.

Recommended:

`https://tourist.example.com/m/{mapId}/p/{placeId}`

This URL represents the destination permanently.

QR workflow:

```
Tourist selects place
↓
Press Show QR
↓
QR appears
↓
Tourist scans QR
↓
Mobile destination page
↓
Tourist chooses Directions
↓
Google Maps / compatible navigation
```

Benefits:

Analytics
Language detection
Destination updates
Apple Maps later
Google Maps
Walking navigation
Campaign tracking
Expired content control

QR codes therefore remain valid even if navigation implementation changes.

24. Smartphone Page

Mobile page:

[Place Image]		
Yodobashi Kyoto		
Short description		
 1.2 km		
[Open Directions]		
[Google Maps]		

Navigation:

```
Current Smartphone Location
↓
Destination Coordinates
↓
Google Maps route
```

25. Map Provider Architecture

Define a generic map provider interface.

MapProvider

└─ GOOGLE

└─ MAPBOX
└─ future providers

Do not couple business logic directly to Google Maps.

Common functions:

```
initializeMap()  
setCenter()  
setZoom()  
setBounds()  
addMarker()  
removeMarker()  
selectMarker()  
fitBounds()  
setMapStyle()
```

26. Map Style

Map style should be separate from provider.

```
ROAD  
SATELLITE  
HYBRID  
TERRAIN  
CUSTOM
```

Client configuration example:

```
Provider:  
Google Maps
```

```
Style:  
Road
```

Future:

```
Provider:  
Mapbox
```

```
Style:  
Client custom tourism style
```

27. Map Range

Two modes:

```
BOUNDED  
UNBOUNDED
```

Bounded

Used for:

```
Station  
Hotel area  
Tourism district  
Shopping facility
```

City district

Settings:

centerLatitude
centerLongitude
defaultZoom

northBoundary
southBoundary
eastBoundary
westBoundary

Unbounded

Allows normal wider map browsing.

28. Signage Map Interaction Policy

For kiosk usage:

allowPan
allowZoom
allowRotation
allowTilt

Recommended default:

allowPan = limited
allowZoom = limited
allowRotation = false
allowTilt = false

When marker selected:

Animate camera
Center marker
Zoom to place
Open side panel

When closed:

Return to configured default camera

29. Search

Tourist search:

Search nearby

Search should primarily search published client content.

Example:

"ramen"

Returns:

client places matching ramen

Google Places live search can be added as an optional feature later.

This prevents uncontrolled external place results from appearing on a curated client map.

30. Branding

Client can configure:

Logo
Map title
Primary theme color
Secondary color
Header appearance
Menu appearance
Font size
Marker style

Do not allow completely arbitrary styling initially.

Use safe configurable presets so signage stays readable.

31. Preview System

Changes must not immediately affect live signage.

Workflow:

```
Client edits
  ↓
Save Draft
  ↓
Preview
  ↓
Publish
  ↓
Published Configuration created
```

Preview should reproduce the Android UI as closely as possible in the web browser.

32. Publish System

This is a critical architectural component.

Do not make Android read editable CMS records directly.

Use:

```
Editable Data
  ↓
Publish Engine
  ↓
Validation
  ↓
Published Snapshot
  ↓
```

Android Signage

Example:

Map Version 41
↓
Client publishes
↓
Map Version 42

Android:

Local Version: 41
Server Version: 42

↓
Download 42
↓
Validate
↓
Cache locally
↓
Activate 42

33. Offline Operation

Public signage should not become unusable if the internet drops.

Android stores the latest valid published map locally.

Cached:

Map configuration
Places
Categories
Translations
Menus
Theme
Images
QR destination information

Online-only features:

Weather
Currency
Live cameras
Real-time events
Analytics uploads

Startup logic:

Internet available?
├── Yes
│ └── Check update
└── No
 └── Load last cached map

34. Android Device Registration

Every signage device receives:

```
deviceId  
customerId  
mapId  
deviceName
```

Example:

```
deviceId:  
dev_kyoto_entrance_01
```

```
customerId:  
cust_jrwest
```

```
mapId:  
map_kyoto_station
```

35. Android Kiosk Mode

The Android application should support:

```
Fullscreen  
Hide navigation bar  
Hide status bar  
Prevent accidental exit  
Auto-launch after reboot  
Screen wake management  
Network recovery  
Automatic map reset
```

Future managed deployment can use Android Enterprise or device-management tooling.

36. Device Heartbeat

Even before a full Super Admin portal exists, design the database for monitoring.

Android periodically reports:

```
deviceId  
status  
lastSeen  
appVersion  
mapVersion  
networkStatus  
lastSync
```

Example:

```
KYOTO-ENTRANCE-01  
ONLINE  
App 1.0.4  
Map v42  
Last Seen 16:24
```

37. Stage 1 Firestore Structure

```
customers/  
  {customerId}  
  
users/  
  {uid}  
  
maps/  
  {mapId}  
  
maps/{mapId}/places/  
  {placeId}  
  
maps/{mapId}/categories/  
  {categoryId}  
  
maps/{mapId}/events/  
  {eventId}  
  
maps/{mapId}/menu/  
  {menuItemId}  
  
maps/{mapId}/languages/  
  {languageId}  
  
maps/{mapId}/widgetInstances/  
  {widgetInstanceId}  
  
maps/{mapId}/liveCameras/  
  {cameraId}  
  
devices/  
  {deviceId}  
  
publishedMaps/  
  {mapId}  
  
publishedMaps/{mapId}/versions/  
  {versionId}
```

Later Super Admin adds:

```
widgetDefinitions/  
themes/  
mapStyles/  
plans/  
features/  
platformVersions/
```

38. Ownership Rules

Every map record must contain:

```
customerId
```

Every client write operation must verify:

```
authenticatedUser.customerId  
==  
resource.customerId
```

A client must never access another client's:

Maps
Places
Events
Devices
Files
Settings

39. Firebase Security

Security should rely on authenticated identity and custom claims / backend validation.

Example roles:

SUPER_ADMIN
CLIENT_ADMIN
CLIENT_EDITOR

Tourist-facing published content is read-only.

Editable admin content requires authentication.

Never depend on hidden UI buttons for authorization.

40. Draft vs Published Data

Recommended conceptual states:

DRAFT
PUBLISHED
ARCHIVED

Content can additionally use:

ACTIVE
INACTIVE
SCHEDULED
EXPIRED

Example event:

```
status = SCHEDULED
startAt = ...
endAt = ...
```

41. Stage 1 Events

Events can be implemented after basic places but before Stage 1 is completely closed.

Event:

eventId
title
description
image
location
startAt

endAt
category
translations
status

The event automatically disappears after expiration.

42. Live Camera

Initial implementation:

YouTube URL

Client fields:

Title
Description
Thumbnail
YouTube URL
Location
Enabled

If a client does not install/enable Live Camera:

No Live Camera menu appears

Future:

YouTube Live
HLS
WebRTC
Direct managed streams

43. Stage 2 — Advanced Client Features

After Stage 1 is stable, expand the Client Admin and Android system.

Stage 2 includes:

Advanced event scheduling
Live camera
Mapbox
Satellite maps
Advanced bounded maps
Custom markers
Client themes
Advanced menu builder
Device management
Analytics
Widget system
Drag-and-drop layout
Weather
Temperature
Currency
Clock

44. Widget Architecture

Widgets must be definitions plus per-map instances.

Example widget definition:

```
Weather Widget
version: 1.0
type: WEATHER
```

Client installation creates:

```
widgetInstanceId
widgetDefinitionId
mapId
customerId
```

```
configuration
position
size
enabled
```

45. Widget Position

Do not store absolute screen pixels.

Use normalized coordinates.

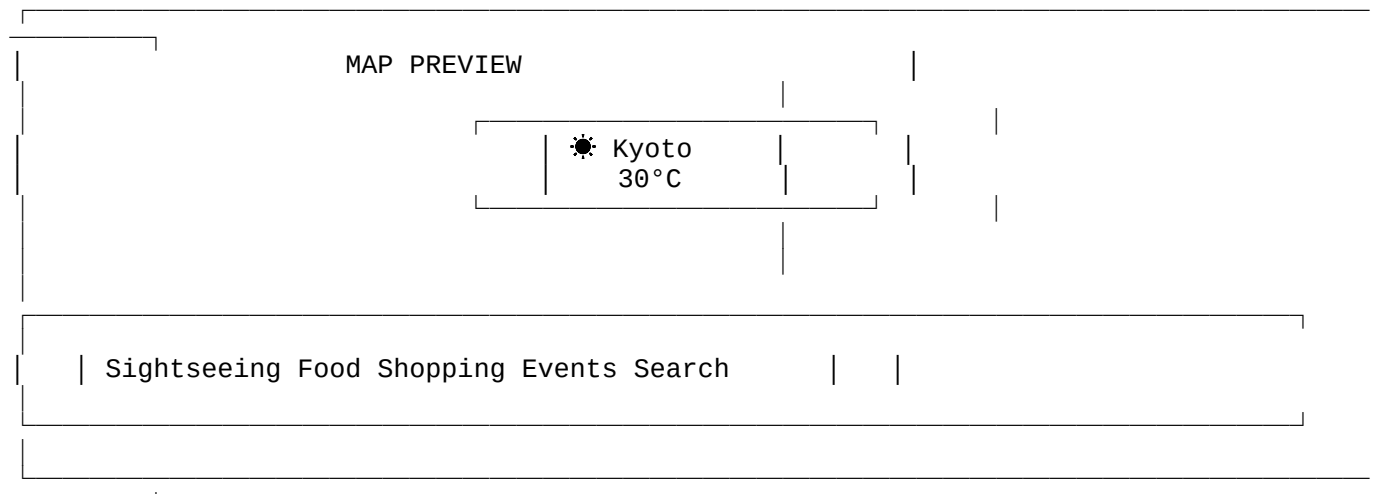
Example:

```
x = 0.78
y = 0.08
width = 0.18
height = 0.12
```

This works on multiple signage resolutions.

46. Widget Editor

Client Map Builder:



Widget Library

Weather	Add
Temperature	Add
Currency	Add
Clock	Add

Client can drag and resize compatible widgets.

47. Stage 3 — Super Admin Platform

After Client + Tourist product is commercially stable, develop Super Admin.

Main modules:

Dashboard

Customers

Users

Maps

Devices

Store

- Widgets
- Themes
- Map Styles
- Components
- Features

Platform

- API Providers
- Feature Flags
- Android Releases
- Web Releases
- Logs
- Monitoring

Analytics

Plans / Billing

48. Widget Store Workflow

Your company develops:

Weather Widget v1.0

Workflow:

Development



Internal Test



Super Admin Draft



Publish to Store



Client sees Weather



Install



Configure
↓
Place on map
↓
Publish Map
↓
Android displays Weather

49. Feature Store

The Store should eventually support more than widgets.

Store item types:

WIDGET
THEME
MAP_STYLE
MENU_COMPONENT
FEATURE
INTEGRATION

Examples:

Weather Widget
Currency Widget
Emergency Information
Train Status
Tourism Theme
Christmas Theme
Map Marker Pack
Live Camera Module
AI Voice Module

50. Super Admin Version Management

Every store item has:

id
name
type
version
status
minimumAppVersion
configurationSchema
releaseNotes

States:

DRAFT
TESTING
PUBLISHED
DEPRECATED

This prevents a newly developed widget from unexpectedly breaking client systems.

51. Android Release Compatibility

Published map configuration should declare requirements.

Example:

```
minimumAndroidAppVersion:  
1.3.0
```

If a client installs a newer widget requiring Android 1.3:

```
Device 1.2  
↓  
Do not publish incompatible configuration
```

The Client Admin should show a compatibility warning.

52. Analytics

Important later analytics:

- Marker selections
- Category selections
- Search terms
- Voice guide plays
- QR displays
- QR scans
- Popular places
- Languages used
- Session duration
- Events opened

Example:

```
Kyoto Station  
Today
```

```
Yodobashi  
Marker Views: 345  
Voice Plays: 102  
QR Scans: 84
```

This can become an important selling feature for municipalities and tourism organizations.

53. Privacy

Avoid collecting unnecessary tourist personal data.

Analytics should generally use:

- Anonymous session ID
- Map ID
- Place ID
- Action
- Timestamp

Do not require a tourist account.

Precise smartphone location should be used only when needed by the smartphone navigation experience and should not automatically be permanently stored.

54. Stage Development Roadmap

STAGE 1A

Project Foundation
Firebase
Multi-tenancy
Registration
Login



STAGE 1B

Client Map CMS
Maps
Places
Google Places
Categories
Images



STAGE 1C

Android Tourist Map
Map
Photo markers
Bottom navigation
Place selection



STAGE 1D

Place Detail Panel
Description
Images
Language
Audio Guide



STAGE 1E

QR / Smartphone Navigation
Dynamic QR
Mobile destination page
Google Maps directions



STAGE 1F

Preview / Publish
Versioning
Android sync
Offline cache



STAGE 1G

Production Signage
Kiosk mode
Device registration
Heartbeat
Error handling
Security

↓

STAGE 1 RELEASE
First Commercial MVP

↓

STAGE 2
Events
Live Camera
More map providers
Themes
Advanced customization
Widgets
Analytics

↓

STAGE 3
Super Admin
Widget Store
Feature Store
Platform Management
Billing

55. Stage 1 Definition of Done

Stage 1 is complete when:

A real client can register

↓

Log into Client Admin

↓

Create/configure a map

↓

Add Google/custom locations

↓

Choose categories

↓

Add multilingual descriptions

↓

Configure branding/menu

↓

Preview

↓

Publish

↓

Physical Android signage receives map

↓

Tourist touches photo marker

↓

Side information panel opens

↓

Tourist hears voice guide

↓

Tourist displays QR

↓

Scans using smartphone

↓

Opens navigation from current location

At this point the system is a usable and sellable Tourist Map MVP.

56. Recommended First Demo

A strong first internal/customer demo should use one real location.

Example:

Kyoto Station Tourist Map

Add approximately:

10-20 locations

such as:

Kyoto Tower
Yodobashi Kyoto
Restaurants
Hotels
Sightseeing
Transport
Shopping

Languages:

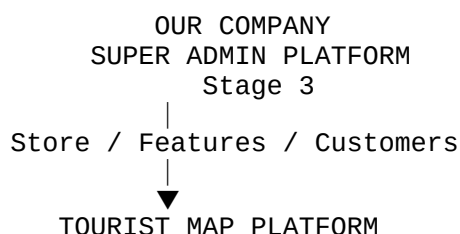
English
Japanese
Chinese

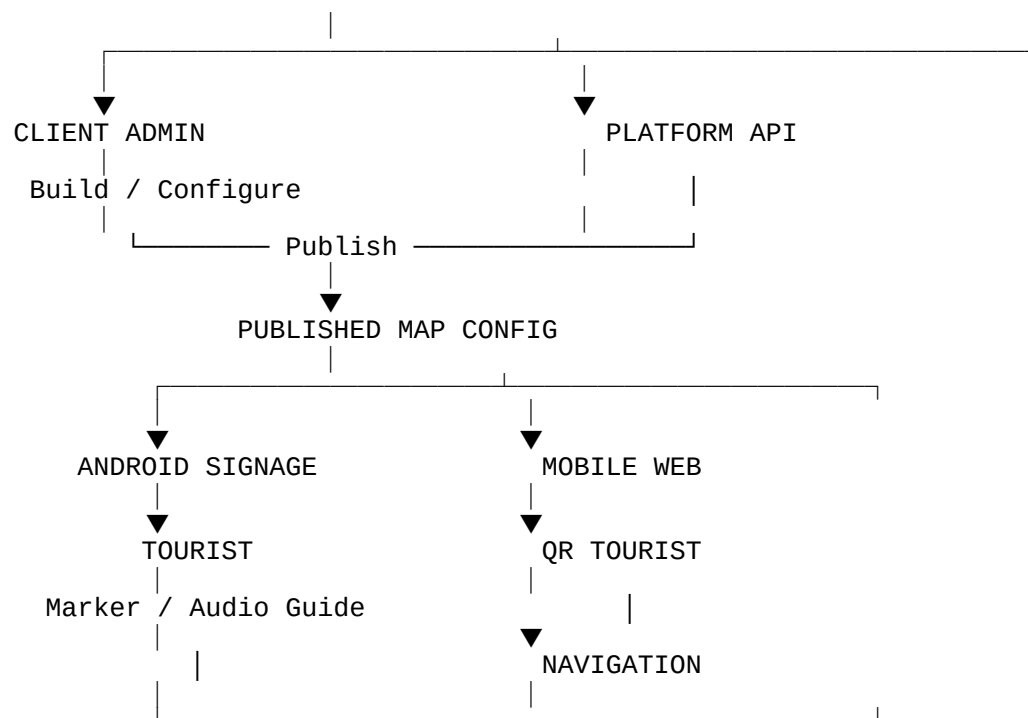
Demonstrate:

Client Login
↓
Add Yodobashi
↓
Add image/description
↓
Publish
↓
Android map updates
↓
Touch Yodobashi photo marker
↓
Left detail panel
↓
English Voice Guide
↓
QR
↓
Scan with smartphone
↓
Google Maps walking directions

That single workflow demonstrates the commercial value of the entire Stage 1 platform.

57. Final Product Architecture





Product Principle

The client does not receive a custom-coded Android application.

The client receives a **configured instance of the Tourist Map Platform**.

One Platform
 One Android Application
 One Client Admin System

+

Different customerId
 Different mapId
 Different content
 Different branding
 Different menu
 Different languages
 Different features

=

Many customer deployments

This is the core architecture that allows the product to scale from one pilot installation to JR, hotels, municipalities and large numbers of signage devices without maintaining separate software projects for every customer.